CS 4475 Fall 2026
Sebastian Vargas
Svargasrivera@miners.utep.edu

Lab Report
Submitted on 9/23/2026

PA2: Priority-based Scheduler for xv6

Task 1. (15 pts) Implement the getprocs() system call and test it using the provided ps command.

File

Kernel/pstat.h

added

```
//////////////////////////////////////////////////
struct pstat {
 int pid; // Process ID
 enum procstate state; // Process state
 uint64 size; // Size of process memory (bytes)
 int ppid; // Parent process ID
 char name[16]; // Parent command name
};
//////////////////////////////////////////////////
```

Where (below)
At the end of the file.
Why
defines the structure to pass information from kernel to user programs

added

```
//////////////////////////////////////////////////
#ifndef _PSTAT_H
#define _PSTAT_H
enum procstate { UNUSED, USED, SLEEPING, RUNNABLE, RUNNING, ZOMBIE
};
//////////////////////////////////////////////////
…
#endif
```

Where (below)
At the top of the file, and at the end of the file respectively.
Why
Moved to allow user programs to use enum procstate, allowing both user and kernel to use the headers.

File

user/user.h

added

```
////////////////
struct pstat;
//////////////////////                              …
////////////////////////////
int getprocs(struct pstat*);
//////////////////////////////////
```

Where (below)
At the top of the file.
…
Bellow the system calls (below the PA1 added wait2)
Why
To declare the variable struct pstat before the file uses it.
…
To declare the system call for the user program, returning an integer (number of processes).

File

user/usys.pl

added

```
##########################
entry("getprocs");
##########################
```

Where (below)
Bellow the system calls (below the PA1 added wait2)
Why
Allows the user to call the new kernel getprocs

File

kernel/syscall.h

Added

```
//////////////////////////
#define SYS_getprocs 24
//////////////////////////
```

where (below)
Bellow the last ID# call (below the PA1 added wait2)
Why
Every SYS_ needs a specific number, since i created SYS_getprocs , it needs the next number (24)

File

kernel/syscall.c.

Added

```
//////////////////////////////
extern uint64 sys_getprocs(void);
//////////////////////////////
…
//////////////////////////////
  [SYS_getprocs] = sys_getprocs,
//////////////////////////////
```

where (below)
Bellow the previously added lines added on PA1 (wait2)
Why
Declares function sys_getprocs exists so it can be referenced.
…
Connect Syscall to the C function that handles them.

File

kernel/defs.h

Added

```
//////////////////////////////
int             getprocs(uint64);
//////////////////////////////
```

where (below)
Bellow the previously added lines added on PA1 (wait2)
Why
Connects sysproc.c with getprocs() allowing it to see that method

File

kernel/sysproc.c

Added

```
//////////////////////////////////
uint64
sys_getprocs(void)
{
  // holds address of the user's array of struct pstat
  uint64 addr;
  // gets first argument from the user (array address), and stores
it into addr>
```

```
  argaddr(0, &addr);
  // calls the kernel function from proc.c, and returns number of
processes.
  return getprocs(addr);
}
////////////////////////////////////////
```

where (below)

sys_wait2(void) method

Why

This is the System call entry point, when getprocs() is executed.

File

kernel/proc.c

Added

Added logic for getprocs

```
//////////////////////////////////////////////////////////////////////
// Fill in an array of struct pstat, located in the caller's
// memory at address addr.
// Return the number of processes, or -1 if an error occurred.
int
getprocs(uint64 addr)
{
  // pointer to current slot of the process table
  struct proc *p;
  // the process that called getprocs.
  struct proc *me = myproc();
  // temp storage for one process's info.
  struct pstat ps;
  // number of processes found so far.
  int proc_found = 0;

  // go through the whole process table
  for (p = proc; p < &proc[NPROC]; p++) {
    // wait_lock is needed to read p->parent safely, and thus needed
before p->lock
    acquire(&wait_lock);
    acquire(&p->lock);

    // if the slot is UNUSED, release both locks and skip it
    if (p->state == UNUSED) {
      release(&p->lock);
      release(&wait_lock);
```

```
      continue;
    }

    // else, it copies current process's info into the temp struct
    ps.pid = p->pid;
    ps.state = p->state;
    ps.size = p->sz;

    // if process has a parent, store parent's pid in ppid (parent
process id)
    if (p->parent)
      ps.ppid = p->parent->pid;
    // else default it to 0
    else
      ps.ppid = 0;
    // copy the process's name into the temp struct, limited to the
size of the name field (16 bytes)
    safestrcpy(ps.name, p->name, sizeof(ps.name));
    // release both locks before copyout and write to user memory
    release(&p->lock);
    release(&wait_lock);
    // copy temp struct to the next available slot of the user's
array,
    // Copyout copies from kernel to user memory
    // me->pagetable, tells the kernel where the user's virtual
address are located on phys mem
    // me->sz, size of caller's memory, used as a bound to prevent
the kernel writing
    // in memory the caller does not own
    // addr + proc_found * sizeof(ps), calculates the destination
address in user memory;
    // addr being the start, sizeof(ps) is the size of the entry,
and by multiplying
    // with proc_found, it skips already written entries; 0 for
first slot, and so on.
    // (char *)&ps, address of temp struct in the kernel, (char *)
to copy raw bytes
    // sizeof(ps), the bytes to copy
    // if the copy fails (less than 0), return -1 as an error
    if (copyout(me->pagetable, me->sz, addr + proc_found *
sizeof(ps),
                (char *)&ps, sizeof(ps)) < 0)
      return -1;
    // count this process
```

```
    proc_found++;
  }
  // number of processes copied to the user's array
  return proc_found;
}
////////////////////////////////////////////////
```

Where (below)
Below method kwait2()
Why
getprocs() allows to get the number of processes on a table.

File

kernel/proc.h

added

```
#include "pstat.h"
…
////////////////////////////////////////////////////
// enum procstate { UNUSED, USED, SLEEPING, RUNNABLE, RUNNING,
ZOMBIE };
////////////////////////////////////////
```

Where (below)
At the top of the file
…
Made it a comment (modify it)
Why
Since enum procstate was moved, it was removed from here.

File

Makefile

added

```
$U/_ps\
```

Where (below)
below $U/_time\
Why
Puts ps in the makefile to be able to use it.

File

user/ps.c

added

```
#include "kernel/param.h"
```

```
#include "kernel/types.h"
#include "kernel/pstat.h"
#include "user/user.h"
int
main(int argc, char **argv)
{
  struct pstat uproc[NPROC];
  int nprocs;
  int i;
  char *state;
  static char *states[] = {
    [SLEEPING]  "sleeping",
    [RUNNABLE]  "runnable",
    [RUNNING]   "running ",
    [ZOMBIE]    "zombie  "
  };
  nprocs = getprocs(uproc);
  if (nprocs < 0)
    exit(-1);
  printf("pid\tstate\t\tsize\tppid\tname\n");
  for (i=0; i<nprocs; i++) {
    state = states[uproc[i].state];
    printf("%d\t%s\t%ld\t%d\t%s\n", uproc[i].pid, state,
                    uproc[i].size, uproc[i].ppid, uproc[i].name);
  }
  exit(0);
}
```

Where (below)
Created the file
Why
calls getprocs() and prints the process table.

Test

```
xv6 kernel is booting

init: starting sh
$ ps
pid     state           size    ppid    name
1       sleeping        16384   0       init
2       sleeping        20480   1       sh
3       running         16384   2       ps
$
```

Ps output.

General workflow

1) ps.c (main) calls getprocs() from user.h
2) usys.pl loads syscall.h (SYS_getprocs())
3) trap.c passes the call to syscall.c
4) syscall.c runs sysproc.c
5) sysproc.c calls proc.c
6) proc.c fills out struct pstat from pstat.h
7) count returns to ps.c, printing the table.

Task 2. (15 pts) Modify the provided ps command to print the priority of each process.

File

kernel/proc.h

added

```
//////////////////////////////////////////////////////////////////
  int priority;                  // Scheduling priority (0 to 49)
//////////////////////////////////////////////////////////////////
```

Where (below)
Inside the struct proc, below int cputime;
Why
Stores the priority of each process

File

kernel/proc.c

added

```
//////////////////////////////////////////////////////
  p->priority = 0;  // Initialize priority
//////////////////////////////////////////////////////
```

Where (below)
Inside allocproc(), below p->cputime = 0;
Why
Initialize priority.

added

```
  ////////////////////2
  np->priority = p->priority;  // Child inherits parent's priority
  ////////////////////////2
```

Where (below)
inside kfork()
Why Allows the child process to inherit the parents priority.

Added

Added logic for k-set and get prioriority

```
//////////////////////////////////////2.2
// Return the calling process's priority.
int
kgetpriority(void)
{
  // myproc() returns a pointer to the process that made the system
call
  struct proc *p = myproc();
  // temp variable to hold the priority while the lock is held.
  int pr;


  // take process's lock.
  acquire(&p->lock);
  // copy the priority into the temp variable.
  pr = p->priority;
  // release the lock.
  release(&p->lock);
  // return the saved value;
  return pr;
}

// Set the calling process's priority. Valid range is 0 to 49.
// Return 0 on success, -1 if out of range.
int
ksetpriority(int pr)
{
  // myproc() returns a pointer to the process that made the system
call
  struct proc *p = myproc();

  // check range (0 to 49)
  if(pr < 0 || pr > 49)
    // if not in range, return -1
    return -1;

  // take process's lock.
  acquire(&p->lock);
  // store the new priority in the process.
  p->priority = pr;
```

```
  // release the lock.
  release(&p->lock);
  // return 0, meaning success.
  return 0;
}
////////////////////////////////////2.2
```

Where (below)
Below method getprocs()
Why
functions that read and change process priority

added

```
    ////////////////////////////////2.2
    // copy the process's priority into the temp struct
    ps.priority = p->priority;
    ////////////////////////////////2.2
```

Where (below)
Inside getprocs()
Why
copies each process’s priority into the struct pstat

File

user/ps.c

added

```
  ////////////////////////////////2.2
 //  printf("pid\tstate\t\tsize\tppid\tname\n");
  printf("pid\tstate\t\tsize\tppid\tname\t\tpriority\n");
  ////////////////////////////////2.2
…
    ////////////////////////////////2.2
    printf("%d\t%s\t%ld\t%d\t%s\t\t%d\n", uproc[i].pid, state,
                    uproc[i].size, uproc[i].ppid, uproc[i].name,
                    uproc[i].priority );
  //  printf("%d\t%s\t%ld\t%d\t%s\n", uproc[i].pid, state,
   //                 uproc[i].size, uproc[i].ppid, uproc[i].name);
   ////////////////////////////////2.2
```

Where (below)
Inside both printf(s) in main()
Why
Now prints the priority in the table

File

kernel/defs.h

Added

```
//////////////////////////2.2
int              kgetpriority(void);
int              ksetpriority(int);
//////////////////////////////2.2
```

where (below)
Bellow the previously added lines added on task 1 (getprocs())
Why
Connects sysproc.c with the setter and getter of priority allowing it to see those methods.

File

kernel/sysproc.c

Added

```
//////////////////////////////////////2.2
// Gets the calling process's priority.
uint64
sys_getpriority(void)
{
  // call kgetpriority() in proc.c and return its result
  return kgetpriority();
}
// Sets the calling process's priority.
uint64
sys_setpriority(void)
{
  // holds the priority value passed by the user program
  int pr;
  // argint allows to read the n-th argument (in this case the
first)
  // and store it in pr
  argint(0, &pr);
  // call ksetpriority() in proc.c and return its result
  return ksetpriority(pr);
}
//////////////////////////////////////2.2
```

where (below) sys_getprocs() method
Why
This is the System call entry point, when either the set or get priority methods are executed.

File

kernel/syscall.h

Added

```
////////////////////////////2.2
#define SYS_getpriority 25
#define SYS_setpriority 26
////////////////////////////2.2
```

where (below)
Bellow the last ID# call (below the PA2 Task 2, getprocs())
Why
Every SYS_ needs a specific number, since I created SYS_getpriority and SYS_setpriority, they need the next number (25 and 26 respectively)

File

kernel/syscall.c.

Added

```
//////////////////////////2.2
extern uint64 sys_getpriority(void);
extern uint64 sys_setpriority(void);
//////////////////////////2.2
…
///////////////2.2
[SYS_getpriority] = sys_getpriority,
[SYS_setpriority] = sys_setpriority,
////////////////////2.2
```

where (below)
Bellow the previously added lines added on PA1 (getprocs)
Why
Declares function sys_getpriority sys_setpriority and exists so it can be referenced.
…
Connect Syscall to the C function that handles them.

added

```
#include "types.h"
#include "param.h"
#include "memlayout.h"
#include "riscv.h"
#include "spinlock.h"
#include "proc.h"
#include "syscall.h"
#include "defs.h"
```

Where (below)
On top
Why
Reformatted to fix syntax error, Each include now is in a separate line.

File

user/usys.pl

added

############################2.2
entry("getpriority");
entry("setpriority");
############################2.2
Where (below)
Bellow the system calls (below the PA2 task 2 added getprocs)
Why
Allows the user to call the new kernel getpriority and setpriority

File

user/user.h

added

//////////////////////2.2
int getpriority(void);
int setpriority(int);
//////////////////////2.2
Where (below)
Bellow the system calls (below the PA2 task 1 added getprocs)
Why
To declare the system call for the user program, returning an integer (for both set and get).

File

kernel/pstat.h

added

////////////////////////////////2.2
int priority; // Scheduling priority
////////////////////////////////2.2
Where (below)
Inside struct pstat
Why
Allows getprocs() to pass priority from the kernel to user programs.

File

user/t2test.c

added

```
////////////////////////////////////2.2
#include "kernel/types.h"
#include "user/user.h"

int
main(void)
{
  // print the priority the process starts with
  printf("initial priority: %d\n", getpriority());

  // change the priority to 20
  setpriority(20);
  // print current priority (20)
  printf("after setpriority(20): %d\n", getpriority());

  // child process should inherit the parent's priority
  if(fork() == 0){
    printf("child inherited: %d\n", getpriority());
    exit(0);
  }
  // wait for the child to finish before the parent continues
  wait(0);

  // 50 is out of range, so it should result in -1
  printf("setpriority(50) returns: %d (expected -1)\n",
setpriority(50));

  // priority must be unchanged after the failed call
  printf("priority after failed set: %d (expected 20)\n",
getpriority());

  exit(0);
}
////////////////////////////////////2.2
```

Where (below)

created file

Why

Test case for Task 2

File

Makefile

added

```
    $U/_t2test\
        $U/_pexec
```

Where (below)
inside UPROGS
Why
Added the task 2 test file and pexec.c to makefile

File

user/pexec.c

added

```
#include "kernel/param.h"
#include "kernel/types.h"
#include "user/user.h"
#define MAXARGS 16

int
main(int argc, char *argv[])
{
    int i, rc;
    char *newargv[MAXARGS];
    int priority;
    if(argc < 3){
        printf("Usage: pexec <priority> <command> [args...]\n");
        exit(-1);
    }
    priority = atoi(argv[1]);
    setpriority(priority);
    for(i=2; i<argc; i++)
    newargv[i-2] = argv[i];
    newargv[argc-2] = 0;
    rc = fork();
    if (rc == 0)
        exec(newargv[0], newargv);
    wait(0);
    exit(0);
}
```

Where (below) Created the file
Why Allows to set priority with argument during execution

getprocs() changes summary

In pstat.h: added a new priority filed in struct pstat
In proc.c: getprocs() copies the priority into the field.

General workflow

1) pexec.c call set/get-priority from user.h
2) usys.pl loads SYS_getpriority/SYS_setpriority
3) trap.c passes the call to syscall.c
4) syscall.c runs sysproc.c
5) sysproc.c calls proc.c
6) proc.c gets or sets p->priority.
7) ps.c calls getprocs(), copying p->priority to struct pstat and prints it.

Tests

```
$ pexec 10 ps
pid     state           size    ppid    name            priority
1       sleeping        16384   0       init            0
2       sleeping        20480   1       sh              0
28      sleeping        16384   2       pexec           10
29      running         16384   28      ps              10
$ pexec 25 sleep 200 &; ps
pid     state           size    ppid    name            priority
1       sleeping        16384   0       init            0
2       sleeping        20480   1       sh              0
30      running         16384   2       ps              0
32      sleeping        16384   1       pexec           25
33      sleeping        16384   32      sleep           25
$ t2test
initial priority: 0
after setpriority(20): 20
child inherited: 20
setpriority(50) returns: -1 (expected -1)
priority after failed set: 20 (expected 20)
$
```

Running pexec.c and t2test(which the child keeps the parent's priority).

Task 3. (15 pts) Add a readytime field to struct proc, initialize it correctly, and modify ps to print a process's age.

File

kernel/proc.h

added

```
//////////////////////////2.3
  uint readytime;            // Tick when the process became
RUNNABLE
//////////////////////////////2.3
```

Where (below)
Inside the struct proc, below int priority;
Why
Stores the tick where process became RUNNABLE, to calculate age.

File

kernel/proc.c

added

```
//////////////////////////2.3
  p->readytime = 0;  // Initialize readytime
////////////////////////////////2.3
```

Where (below)
Inside allocproc(), below p->priority = 0;
Why
Initialize readytime.

added

```
//////////////////////////2.3
      p->readytime = ticks;  // Process becomes RUNNABLE
////////////////////////////////2.3
…
//////////////////////////2.3
np->readytime = ticks;  // Process becomes RUNNABLE
////////////////////////////////2.3
```

Where (below)
Inside wakeup(), below p->state = RUNNABLE; Inside kkill(),below p->state = RUNNABLE;
Inside userinit(), below p->state = RUNNABLE; inside yield(), below p->state = RUNNABLE;
…
inside kfork(), below np->state = RUNNABLE;
Why
Get ticks where it becomes RUNNABLE

File

kernel/pstat.h

added

```
//////////////////////////2.3
// copy the process's readytime into the temp struct
    ps.readytime = p->readytime;
////////////////////////////////2.3
```

Where (below)
Inside getprocs()
Why
copies each process’s readytime into the struct pstat

File

user/ps.c

added

```
 ////////////////////////////////////2.2, 2.3
  //  printf("pid\tstate\t\tsize\tppid\tname\n");
  // printf("pid\tstate\t\tsize\tppid\tname\t\tpriority\n");
  printf("pid\tstate\t\tsize\tage\tppid\tname\t\tpriority\n");
  ////////////////////////////////2.2, 2.3
…
    ////////////////////////////////////2.2, 2.3
    printf("%d\t%s\t%ld\t%d\t%d\t%s\t\t%d\n", uproc[i].pid, state,
                    uproc[i].size,age, uproc[i].ppid, uproc[i].name,
                  uproc[i].priority );
   // printf("%d\t%s\t%ld\t%d\t%s\t\t%d\n", uproc[i].pid, state,
   //                 uproc[i].size, uproc[i].ppid, uproc[i].name,
   //               uproc[i].priority );
   //  printf("%d\t%s\t%ld\t%d\t%s\n", uproc[i].pid, state,
   //                 uproc[i].size, uproc[i].ppid, uproc[i].name);
   ////////////////////////////////////2.2, 2.3
```

Where (below)
Inside both printf(s) in main()
Why
Now prints the age in the table

added

```
 ////////////////2.3
```

```
  int age;
  ////////////////////2.3
…
    /////////////////2.3
    // age is the time spent waiting, when RUNNABLE
    if (uproc[i].state == RUNNABLE)
      // if runnable, calculate age
      age = uptime() - uproc[i].readytime;
    else
      // else default to 0
      age = 0;
    ////////////////2.3
```

Where (below)
Inside for loop, after state = states[uproc[i].state];
Why
Calculates age using readytime

Tests

```
$ ps
pid     state           size    age     ppid    name            priority
1       sleeping        16384   0       0       init            0
2       sleeping        20480   0       1       sh              0
29      running         16384   0       2       ps              0
28      runnable        16384   0       1       matmul          0
$ matmul 5 &; ps
pid     state           size    age     ppid    name            priority
1       sleeping        16384   0       0       init            0
2       sleeping        20480   0       1       sh              0
30      running         16384   0       2       ps              0
28      runnable        16384   4       1       matmul          0
32      runnable        16384   3       1       matmul          0
```

Process that are RUNNABLE are non-zero. Since ready time counts one only for those that are RUNNABLE

Task 4. (15 pts) Implement a priority-based scheduler.

File

kernel/param.h

added

//////////////////////2.4
// Scheduling policies
#define SCHED_RR 0
#define SCHED_PRIORITY 1
// #define SCHEDULER SCHED_RR
#define SCHEDULER SCHED_PRIORITY
//////////////////////2.4

Where (below)

end of file

Why

Commenting respectively either #define SCHEDULER SCHED_RR or #define SCHEDULER SCHED_PRIORITY, to choose which schedule model to use.

File

kernel/proc.c

added

```
//////////////////////2.4
#if SCHEDULER == SCHED_RR
//////////////////////////2.4
…
////////////////////////////////////2.4
#elif SCHEDULER == SCHED_PRIORITY
// Priority scheduler: runs the RUNNABLE process
// with the highest priority.
void
scheduler(void)
{
  // loop variable over the process table
  struct proc *p;
  // holder of best RUNNABLE process so far
  struct proc *best;
  // pointer to the CPU this scheduler runs on
  struct cpu *c = mycpu();

  // no process is running on this CPU yet
  c->proc = 0;
  for (;;) {
    // The most recent process to run may have had interrupts
    // turned off; enable them to avoid a deadlock if all
    // processes are waiting. Then turn them back off
    // to avoid a possible race between an interrupt
    // and wfi.
```

```
    intr_on();
    intr_off();

    // no candidate found yet at the start of each scan
    best = 0;
    // go through the whole process table
    for (p = proc; p < &proc[NPROC]; p++) {
      // lock the slot before reading its state and priority
      acquire(&p->lock);
      // candidate must be RUNNABLE.
      // if it is the first one found (best == 0)
      // or with a higher priority than the current best
      // it is replaced.
      if (p->state == RUNNABLE &&
          (best == 0 || p->priority > best->priority)) {
        // the previous best is replaced, and its lock is released
        if (best != 0)
          release(&best->lock);
        // the new best keeps its lock held.
        best = p;
        // skip the release
        continue;
      }
      // not a better candidate, so release its lock
      release(&p->lock);
    }
    // a RUNNABLE process was found
    if (best != 0) {
      // best is still locked, and can be switched to safely
      p = best;
      // Switch to chosen process.  It is the process's job
      // to release its lock and then reacquire it
      // before jumping back to us.
      p->state = RUNNING;
      // record it as the process running on this CPU
      c->proc = p;
      // switch from the scheduler context to the process context
      swtch(&c->context, &p->context);

      // Don't re-enable interrupts on release.
      mycpu()->intena = 0;
      // Process is done running for now.
      // It should have changed its p->state before coming back.
      c->proc = 0;
```

```
      // release the lock held since the scan
      release(&p->lock);
    }
    else {
      // nothing to run, wait for an interrupt
      asm volatile("wfi");
    }
  }
}
#endif
////////////////////////////////////2.4
```

Where (below)
Above and below the scheduler respectively.
Why
Added logic of the priority scheduler.

Round-robin test

```
xv6 kernel is booting

init: starting sh
$ pexec 5 matmul 5 &; pexec 20 matmul 5 &
$ Time: 310 ticks
Time: 309 ticks
```

Both processes shared the CPU, taking twice as long as a single run.

Priority test

```
xv6 kernel is booting

init: starting sh
$ pexec 5 matmul 5 &; pexec 20 matmul 5 &
$ Time: 161 ticks
Time: 161 ticks
```

Priority 20 process ran to completion, and the priority 5 started afterwards

Task 5. (15 pts) Add aging to your priority-based scheduler.

File

kernel/proc.c

added

```
////////////////////////////2.5
// Effective priority: priority plus the time spent waiting as
RUNNABLE.
```

```
// Capped at 99.
static int
effpriority(struct proc *p)
{
   #if AGING
  // time waiting since the process became RUNNABLE
  int eff = p->priority + (ticks - p->readytime);
  #else
  // aging disabled, so effective priority equals priority
  int eff = p->priority;
  #endif
  // cap at the maximum effective priority
  if (eff > 99)
    eff = 99;
  return eff;
}
////////////////////////////2.5
```

Where (below)
above #if SCHEDULER == SCHED_RR
Why
computes effective priority (the longer a runnable process waits, the higher priority (age)).

added

```
////////////////////////////2.5
         //  (best == 0 || p->priority > best->priority)) {
         (best == 0 || effpriority(p) > effpriority(best))) {
////////////////////////////2.5
```

Where (below)
inside the priority scheduler()
Why
now picks not the best, but the one with highest effective priority.

File

kernel/param.h

added

```
////////////2.5
#define AGING 0
///////////////2.5
```

Where (below)
End of the file
Why
Selects if aging is (1) or is not (0) applied. Hardcode

Tests

For 0 (no aging)

```
xv6 kernel is booting

init: starting sh
$ time pexec 5 matmul 5 &; time pexec 20 matmul 5 &
$ Time: 173 ticks
Time: 168 ticks
elapsed time: 342 ticks, cpu time: 0 ticks, 0% CPU
elapsed time: 343 ticks, cpu time: 1 ticks, 0% CPU
```

For 1 (with aging)

```
xv6 kernel is booting

init: starting sh
$ time pexec 5 matmul 5 &; time pexec 20 matmul 5 &
$ Time: 166 ticks
elapsed time: 177 ticks, cpu time: 1 ticks, 0% CPU
Time: 318 ticks
elapsed time: 319 ticks, cpu time: 0 ticks, 0% CPU
```

The longer a RUNNABLE process waits, the higher is effective priority gets, allowing low priority processes to eventually run.

Task 6. (5 pts) Summary

I learned better how the priority systems work, especially with aging. Aging prevents processes from staying behind when newer.
I also improved the understanding of the locks and how XV6 uses them.
The difficulties were mainly in task 1 and 2, due to how many modifications were needed, making sure all of them have been properly modified. It was mind breaking keeping track at first, made me stuck on it.
As well I had to change enum procstate to make it work inside user programs (since it was only defined on the Kernel).

Extra Credit Task (10 points).

…